

程设大作业最终报告

Seteer —— 字符串空间卡牌对战游戏

组号：43

成员：杨靖宇、胡薛淇、泽仁罗布

1 程序功能介绍

Seteer 是一款独特的卡牌游戏——除了打牌，你还得学会“玩字符串”。每次使用卡牌，都有可能往一个公共的字符串空间里写入内容：随机字母、数字，甚至是清空它。而这个字符串的形态，又会反过来决定具体牌效和战局走向，这是游戏最核心的乐趣。

游戏流程

打开游戏即为主菜单，点“新游戏”就能立刻开始。
整局游戏开始会进入“祈福”事件：从三个初始祝福中选一个（加力量、抽牌或者每回合自动叠甲等）。战斗层总共四层，每一层你都会面对一个敌人，每层的敌人强度会逐渐提升，每一次胜利之后可以选一张新牌加入牌组，再休息回血，接着挑战下一层，直到击败最终 BOSS。任意行为造成的血量归 0 都会将你送回塔底。

战斗流程

进入战斗，双方自动初始化。（血量继承，祈福生效，能力清除，能量恢复等）每回合我方先行动，默认从手牌堆抽取 5 张手牌（祈福与能力会影响抽取手牌数）将卡牌从手牌中拖入页面中部视为出牌，出牌时会判定能量是否足够，若足够则打出且扣除对应能量（部分卡牌不需要能量即可打出）卡牌有各式各样的效果，其类型大致分为三类：

- 攻击牌（像“打击”、“清空爆破”）：以造成伤害为主要效果的牌，有些还会根据字符串内容来改变决定伤害值并改变字符串。
- 技能牌（像“随机数字”、“灵巧抽牌”）：造成伤害之外的效果的牌，如操作字符串空间、抽牌、回能量，灵活支撑你的策略。
- 能力牌（像“随机触碰”、“护符共鸣”）：不同于前两者，能力牌通常不直接造成效果，而是给予玩家一个“能力”，能力会在某种条件下触发，如每回合开始，每次抽牌，每次写入字符串空间。

除了造成伤害、获得格挡、改变字符串，卡牌效果还包括给予状态、获得力量、抽牌等。玩家和敌人在战斗中会因为不同的原因获得状态或能力。

我方行动结束后，可以点击结束回合按键以结束回合。之后轮到敌方行动，敌方将会以意图结合战斗实时情况进行行动。每一种敌人都有自己的行为模式，有些强力的敌人甚至具有多个阶段，需要小心应对。

敌方行动完后，该回合结束，双方的格挡清零，我方能量恢复如此循环，直到任意一方生命值归 0。

字符串空间就显示在画面中央，你随时可以看见里面的内容（最长 20 个字符）。很多牌都有条件效果：“如果末尾是数字”、“如果末尾是字母”、“如果字符串长度 > 2”。有效利用字符串空间往往带来事半功倍的效果。

ui 与显示优化

手牌拖到屏幕中部算作打出，拖到下面就收回，防止误操作。对于我方和敌方，均有模型，血条，状态栏等基本 ui，血条会根据剩余血量占比和是否持有格挡变换颜色。我方能量槽显示剩余能量数，以及抽牌堆和弃牌堆剩余牌数，敌方头顶显示意图。

页面下方显示手牌，手牌上方显示实时的字符串空间；页面右下为结束回合按键，显示回合数；页面左上为已选祈福

过多文本显示会让页面变得拥挤，为此我们设计了悬浮窗口，使得部分复杂效果可以通过悬停在图标上以显示。包括：卡牌的详细条件和效果状态/能力的效果祈福的效果

我们用简单的精灵动画来提升手感：出攻击牌，角色会往前冲一下；挨打会闪红；死亡时会转圈淡出。

2 项目模块与类设计

2.1 战斗系统

战斗系统是整个游戏的核心。它的设计目标其实很简洁：

- 每一回合，玩家和敌人轮流行动；
- 所有行动（卡牌、敌人攻击、被动能力）都用同一套“效果”来描述；
- 当行动发生时，系统要能自动区分“谁在打谁”，并正确处理伤害、格挡、状态等交互。

Entity 类

我们从基本的 Entity 基类的设计开始介绍。

无论是玩家还是敌人，它们都需要管理生命值、格挡值、力量加成和身上的状态（易伤、虚弱）。于是我们抽象出一个 Entity 基类，集中处理所有“战斗单位”的通用属性：

- m_hp / m_maxHp: 当前生命与最大生命
- m_block: 格挡值，用于抵消受到的伤害

- `m_strength`: 力量, 使每次攻击额外造成等于力量值的固定伤害
- `m_statuses`: 状态列表 (如易伤 2 回合)
- `m_abilities`: 能力列表 (被动效果, 稍后详述)

例如, `Entity` 提供了一个统一的受伤接口 `takeDamage(int)`, 代码如下:

Listing 1: `Entity::takeDamage` 实现

```
1 void Entity::takeDamage(int dmg) {
2     if (dmg <= 0 || isDead()) return;
3     if (hasStatus(StatusType::Vulnerable)) {
4         dmg = static_cast<int>(dmg * 1.5);
5     }
6     if (m_block > 0) {
7         int blocked = qMin(dmg, m_block);
8         m_block -= blocked;
9         dmg -= blocked;
10    }
11    m_hp = qMax(0, m_hp - dmg);
12    if (onDamaged && dmg > 0) onDamaged(dmg);
13    if (onHit && dmg > 0) onHit(dmg);
14 }
```

任何实现了 `Entity` 的子类, 只要调用 `takeDamage`, 就能共享这套处理。

玩家与敌人只在“自己独有的能力”上有差异, 因此我们继承 `Entity` 并各自扩展:

玩家 (`Player`) 拥有能量系统:

- `m_energy / m_maxEnergy`: 当前能量与上限 (默认 3, 可被卡牌提升至 4)
- `restoreEnergy()`: 回合开始时回满能量
- `onTurnStart()`: 回能并清空格挡

敌人 (`Enemy`) 则持有一个 AI 对象, 负责决定它每回合要做什么。将 AI 从 `Enemy` 中抽离出来, 是因为不同敌人有着完全不同的行动模式。于是我们定义了 `EnemyAI` 基类, 它的核心方法只有一个:

```
1 virtual std::vector<Effect> decide(const BattleContext& ctx) = 0;
```

每个具体敌人 (如 `Doubao`, `Dragon`) 通过构造函数注入不同的 AI 子类, 敌人本身只负责调用 `decideIntent()`, 将获得的效果列表暂存起来, 等轮到它行动时再执行。

在此, 我们能很自然地引入 `BattleContext` 的设计。AI 在决策时需要知道战场的当前状态——敌人的血量、字符串里的内容、甚至玩家还有多少格挡。于是我们设计了一个轻量级的战场上下文 `BattleContext`, 它把所有回合内需要的环境信息打包成一个结构体:

- `player`: 指向玩家的指针
- `enemy`: 指向敌人的指针
- `stringSpace`: 字符串空间的引用
- `drawPile / hand / discardPile`: 牌堆的引用
- `attacker` 与 `defender`: 当前行动的执行者和目标
- `drawCards`: 一个函数对象, 用于在不依赖 `Game` 类的情况下完成抽牌

这个设计的关键在于 **attacker** 和 **defender**。任何效果在执行前，调用者只需要设置好这两个指针，效果代码本身就能无视“谁在打谁”的差异，同一套效果代码同时适用于玩家和敌人。后面我们会看到，无论是卡牌、AI 决策还是被动能力的触发，所有的效果执行都依赖这一个 **BattleContext**。

Effect 类及其派生

不管是玩家打出的卡牌、敌人执行的攻击，还是被动能力在特定时机触发的效果，底层的“发生什么事”都归结为一个统一的概念——效果（Effect）。

我们希望能够灵活组合各种逻辑，而不是为每张牌写一段专门的代码。因此，一个效果被拆成了两个核心零件：

1. 条件（**Condition**）：决定效果是否触发。可以为空，表示无条件触发。例如“字符串末尾是数字”、“玩家没有格挡”。
 2. 动作（**EffectAction**）：描述具体行为，如造成伤害、获得格挡、抽牌、写入随机字母等。
- 在代码层面，**Effect** 结构体将这两者聚合在一起：

Listing 2: Effect 结构体

```

1 struct Effect {
2     std::unique_ptr<Condition> condition; // nullptr 表示无条件
3     std::unique_ptr<EffectAction> action;
4
5     void execute(BattleContext& ctx) const {
6         if (!condition || condition->evaluate(ctx) != condition->invert)
7             action->execute(ctx); // 条件满足（或无条件）时执行动作
8     }
9 };

```

有了这个框架，卡牌数据只需要配置一个 **Effect** 列表，运行时按顺序遍历执行即可。例如“打击”这张牌的效果配置为：

- 效果 1：无条件 **DamageAction**，造成 6 点伤害。
- 效果 2：**LastCharIsDigitCondition** + **ConsumeLastDigitDamageAction** ——如果字符串末尾是数字，消耗该字符并造成等量伤害。

动作需要的数值（如伤害量、格挡量）目前多数直接以常量形式传入动作的构造函数；少数动作（如 **DamageAction**、**BlockAction**）也接受一个简单的 **EffectValue** 对象来表示固定值或未来的动态值，但现阶段动态取值尚未深度使用，数值基本是固定的。这种“条件 + 动作”的组合方式让设计新卡牌变得极其简单——只需要在数据库里组合已有的条件和动作，几乎不需要写新的逻辑。

有了统一的效果模型之后，敌人的行为就变成了“每回合生成一组 **Effect**”。**EnemyAI** 的 **decide()** 方法返回一个 **std::vector<Effect>**，例如 **DoubaoAI** 的第一回合只需要返回：

```

1 std::vector<Effect> DoubaoAI::decide(const BattleContext& ctx) {
2     std::vector<Effect> effects;
3     effects.push_back(Effect(std::make_unique<DamageAction>(EffectValue::fixed(5))));
4     m_intentDesc = "认知偏差：造成 5 点伤害";

```

```

5     return effects;
6 }

```

Enemy 在回合开始时调用 `decideIntent()` 暂存这些效果，等轮到敌人行动时，Game 层取出效果列表，设置 `attacker = enemy`, `defender = player`，逐个调用 `effect.execute(ctx)` 即可。整个过程和玩家打牌使用的是完全相同的 `Effect::execute` 路径。

敌人的效果是由 AI 在运行时动态生成的，而玩家的卡牌效果是在游戏启动时就配置好的。每张牌的数据 (CardData) 中持有一个 `std::vector<Effect>`，在 `CardDatabase::registerDefaults()` 中一次性初始化。以“打击”为例：

```

1 c->effects.push_back(
2     Effect(std::make_unique<DamageAction>(EffectValue::fixed(6))));
3 c->effects.push_back(
4     Effect(std::make_unique<LastCharlsDigitCondition>(),
5         std::make_unique<ConsumeLastDigitDamageAction>()));

```

玩家出牌时，Game 层遍历这个列表，设置 `attacker = &player`, `defender = &enemy`，依次调用 `effect.execute(ctx)`。条件和动作的判定逻辑与敌人行动完全一致——这就是统一效果模型的核心价值。至此，我们已经看到了效果系统的全貌：它由简单的“条件 + 动作”构成，既能被 EnemyAI 动态生成，也能被卡牌数据静态配置，所有执行路径最终汇聚到同一个 `Effect::execute` 调用。

回合进程

整个战斗被划分成明确交替的两个阶段：玩家回合与敌人回合，由 Game 类统一调度。

玩家回合的入口是 `startPlayerTurn()`。它按顺序完成以下工作：

1. 增加回合计数，应用所有祝福效果（如“每回合获得 2 点力量”）；
2. 调用 `player.onTurnStart()`——回满能量、清空格挡；
3. 触发所有 `OnTurnStart` 时机的能力（如龙的每回合叠甲、随机触碰写入字母）；
4. 倒计时更新状态和能力，移除到期的；
5. 抽 5 张牌。

此后玩家可以自由操作：点击或拖拽手牌来打出，直到主动点击“结束回合”按钮。

玩家出牌时，`playCard` 方法首先检查能量是否足够，然后扣减能量，设置 `attacker = &player`, `defender = &enemy`，遍历该卡牌预配置的 `Effect` 列表逐一执行。如果卡牌带有攻击属性，还会播放攻击动画作为视觉反馈。每打出一张牌后立即检查敌人是否死亡，若死亡则直接进入结算。

敌人回合的入口是 `enemyAction()`。由于敌人行为已经由 `enemyAI` 在上一个玩家回合结束时提前决定好了，这里只需要取出暂存的效果列表，设置 `attacker = &enemy`, `defender = &player`，逐一执行即可。执行前先播放敌人攻击动画，动画结束后再实际调用效果，保证视觉和逻辑同步。

敌人行动完毕后，检查玩家是否死亡：

- 若玩家死亡，播放死亡动画，战斗以失败结束；
- 否则，敌人重新调用 `decideIntent()` 决定下一回合的意图，然后再次进入玩家回合。

整个回合控制的核心逻辑可以概括为：效果是由谁生成的并不重要，执行时统一走 **BattleContext** + **Effect::execute** 这一条路。Game 类的职责仅仅是摆好 `attacker/defender` 的位置，然后按下“执行”按钮。

牌堆管理：抽牌、弃牌与循环利用

卡牌在战斗中的生命周期由三个牌堆共同管理：抽牌堆（`drawPile`）、手牌（`hand`）和弃牌堆（`discardPile`）。

抽牌由 `drawCards(int count)` 实现。它每次从抽牌堆顶部取一张牌加入手牌，同时创建对应的 `CardItem` 图形项用于显示和交互。如果抽牌堆见底，系统会自动将弃牌堆的所有牌移入抽牌堆并重新洗牌，保证抽牌永不中断。这个设计模拟了经典 Roguelike 牌组游戏的手感——你不会因为牌堆耗尽而“无牌可用”，但会经历一轮牌组的循环周期。

弃牌发生在两个时机：玩家回合结束时（`endPlayerTurn` 调用 `discardHand`），以及单张牌被打出后移入手牌列表时（除非该牌带有“消耗”属性，则直接移出战斗）。

抽牌动作的解耦值得一提。`DrawAction` 是效果系统中的一个动作类，它需要触发抽牌逻辑，但不能直接依赖 `Game` 类（否则效果系统就和战斗控制器耦合在一起了）。我们在 `BattleContext` 中放了一个函数对象 `drawCards`，由 `Game::buildContext()` 在每次构建上下文时绑定到自身的 `drawCards` 方法。于是 `DrawAction::execute` 只需调用 `ctx.drawCards(n)` 即可，完全不需要知道 `Game` 的存在。这个技巧同样可以用于未来的其它“需要调用 `Game` 层能力”的动作类。

StringSpace 类

`StringSpace` 是一个长度上限为 20 的可变字符串，在战斗中充当玩家与敌人之间的“共用黑板”。它提供了一组简单的方法：

- `append(text)`：在末尾追加内容；
- `clear()`：清空整个字符串；
- `removeLast(n)`：删除末尾 `n` 个字符；
- `lastChar()`、`isLastDigit()`、`isLastAlpha()`：查询末尾字符的类型。

每当字符串发生修改，`StringSpace` 会分别调用两个回调：`onChanged` 用于刷新 UI 显示，而 `onAppended` 则用来触发能力（如“每次写入内容时获得 3 点格挡”）。所有依赖于字符串空间的卡牌条件和能力触发，都通过访问 `BattleContext` 中的 `stringSpace` 指针来完成，与核心战斗逻辑完全解耦。

Status 相关方法

`StatusType` 枚举定义了易伤（Vulnerable，受到伤害 $\times 1.5$ ）和虚弱（Weak，造成伤害 $\times 0.75$ ）。`StatusInstance` 记录了状态的类型、数值和剩余回合数，存放在 `Entity::m_statuses` 列表中。

核心操作只有两个：

`addStatus` 用于施加新状态或叠加已有状态的持续回合；

`tickStatuses` 在回合结束时被调用，将所有状态的持续回合减 1，到期者自动移除。

除此之外，`Entity::takeDamage` 在每次受伤时会自动检查易伤状态并放大伤害，`DamageAction` 则会在计算攻击时检查攻击者身上的虚弱状态并减小输出。整个过程对外部调用者透明，只需正确设置 `attacker/defender` 即可。

Ability 相关方法

`AbilityData` 描述一个被动能力的静态配置：触发时机（`Trigger` 枚举）、执行的效果动作、可选的触发条件和默认持续回合。`AbilityInstance` 则记录具体实体上叠加的某个能力及其剩余回合（0 表示永久）。

实体在获得能力时，将能力的 ID 和持续回合记录到 `m_abilities` 列表中。每当发生特定事件（回合开始、写入字符串、打出卡牌、回合结束），调用 `triggerAbilities(trigger, ctx)` 就会遍历能力列表，对每个能力从 `AbilityDatabase` 取出数据，匹配触发时机并执行对应的 `EffectAction`。因为能力执行的效果和卡牌使用的 `Effect` 是完全相同的东西，所以“每回合开始时获得 2 点格挡”这样的能力只需配一个 `OnTurnStart + BlockAction`，执行时自动通过 `ctx.attacker` 赋给拥有者。

回合结束时，`tickAbilities` 将所有非永久能力的剩余回合减 1，到期后自动移除，与状态系统的倒计时管理完全对称。

2.2 跑团管理

战斗系统之外，游戏还需要管理整个“跑团”流程——从开局祈福、逐层战斗、事件选择，到最终胜利或失败，这一层由 `RunManager` 统一调度。

`RunManager` 通过维护楼层推进来统筹游戏进程，他维护一个楼层计划（`m_floorPlan`），按顺序依次执行祈福、战斗、选牌、休息等事件。每个事件类型由 `FloorStep::Type` 枚举定义：

Listing 3: 楼层步骤类型

```
1 struct FloorStep {
2     enum Type { Blessing, Battle, CardPick, Rest, End };
3     Type type;
4     QString enemyId;    // 战斗专用
5     int layer;          // 第几层
6 };
```

在 `start()` 中, `RunManager` 生成完整的楼层计划:

1. 祈福 (选择初始祝福);
2. 第 1 层战斗 → 选牌 → 休息;
3. 第 2 层战斗 → 选牌 → 休息;
4. 第 3 层战斗 → 选牌 → 休息;
5. 第 4 层 (BOSS 层) 战斗 → 结束。

每个楼层执行时, `executeFloor` 函数会根据步骤类型做不同处理:

- 祈福: 调用 `generateBlessingOptions()` 从数据库中随机抽取 3 个祝福, 通过信号 `blessingOptionsAvailable` 通知 `MainWindow` 显示选择界面。
- 战斗: 调用 `resolveEnemyId()` 决定敌人 (支持调试指定), 发出 `battleStarting` 信号, 携带敌人 ID 和层数。
- 选牌: 调用 `generateCardPickOptions()` 生成候选牌 (默认池包含各类攻击、技能、能力牌), 发出 `cardPickOptionsAvailable` 信号。
- 休息: 计算回复量 (最大 HP 的 30%), 发出 `restOptionAvailable` 信号。

用户做出选择后, 通过信号槽返回 `RunManager` 的对应槽函数 (如 `onBlessingChosen`), 更新玩家数据 (牌组、祝福、HP), 然后推进至下一楼层。

2.3 UI 与动画架构

整体布局与页面管理

主窗口 `'MainWindow'` 继承自 `'QMainWindow'`, 核心组件是一个 `'QStackedWidget'`, 用于管理三种不同的页面:

1. 主菜单页: 显示 “新游戏”、“继续游戏”、“退出” 按钮, 背景使用游戏主题图。
2. 事件页: 用于非战斗场景 (祈福、选牌、休息), 由 `'RunManager'` 触发并显示对应的选项按钮。
3. 战斗场景: 继承自 `'QGraphicsView'`, 承载完整的战斗界面, 包括精灵、血条、手牌、字符串空间等所有图形元素。
4. 边栏: 位于窗口顶部, 提供存档功能, 方便玩家在事件页手动存档。

`'MainWindow'` 通过信号/槽接收 `'RunManager'` 的事件通知, 动态创建并切换页面, 并在战斗结束后显示胜利或失败图。

战斗场景 (**Game**)

`'Game'` 类是战斗界面的核心, 它继承自 `'QGraphicsView'`, 并管理一个 `'QGraphicsScene'`。`Game` 类中定义了多个布局常量, 确保所有图形元素在不同窗口尺寸下保持相对位置。`'updateUI()'` 方法每帧重新计算精灵、血条、能量条、状态图标等的位置和样式。战斗界面不光显示每张牌的图标, 同时支持手牌的一系列交互行为, 包括:

- 点击: 选中卡牌并略微上浮, 便于玩家确认。

- 拖拽：当鼠标拖动距离超过阈值（8 像素）时，卡牌跟随鼠标移动；若释放位置高于屏幕中部的 `PLAY_ZONE_Y`，则视为打出该卡牌，否则卡牌回到原位。
- 悬停：鼠标悬停时，卡牌放大并显示详细的卡牌描述。

卡牌在场景中的排列由 `Game::rearrangeHand()` 负责，根据手牌数量动态计算间距和起始位置。此外，我们设计了 `TooltipIcon` 类（继承自 `QGraphicsPixmapItem`）用于显示祈福、状态（易伤/虚弱）、能力（如“龙鳞再生”）以及力量等图标。关键特性是悬停：与手牌类似，当鼠标悬停在图标上时，会弹出一个半透明背景的文本气泡，显示该状态的详细说明。

精灵动画（`SpriteAnimator`）

`SpriteAnimator` 是一个独立的动画管理类，负责驱动各种动作。它使用 `QPropertyAnimation` 和 `QSequentialAnimationGroup` 实现以下动画效果：

- 攻击动画：精灵向前冲刺一段距离后返回原位，模拟攻击动作。
- 动作动画（施法/防御）：精灵向上弹起再落下，并伴有轻微缩放，用于非攻击卡牌的使用反馈。
- 受击动画：精灵快速闪烁两次（透明度变化），表示受到伤害。
- 死亡动画：精灵旋转并淡出，同时略微上浮，营造“消散”的视觉效果。

事件页（`EventPage`）

`EventPage` 是一个轻量级 `QWidget`，用于展示三种非战斗事件：

- 祈福：显示三个祝福按钮，每个按钮包含名称和描述。
- 选牌：显示三张可选卡牌按钮，同样包含名称和描述。
- 休息：显示一个“休息”按钮，并告知本次可回复的生命值。

用户点击按钮后，`EventPage` 发出对应的信号（`blessingChosen`、`cardChosen`、`restChosen`），由 `MainWindow` 转发给 `RunManager`。

2.4 模块间信号槽通信

整个程序采用 Qt 的信号槽机制实现模块解耦，核心连接如下：

- `RunManager::blessingOptionsAvailable()` → `MainWindow::onBlessingOptions()`：触发祈福事件页。
- `RunManager::battleStarting()` → `MainWindow::onBattleFloor()`：触发战斗场景创建与切换。
- `Game::battleFinished()` → `MainWindow::onBattleFinished()`：战斗结束，告知结果。
- `EventPage::blessingChosen()` → `MainWindow::blessingSelected()` → `RunManager::onBlessingChosen()`：点击按钮选择祈福。
- `EventPage::cardChosen()`、`EventPage::restChosen()` 同理，分别对应选牌和休息。

- `GameOverMenu::returnToMenu()` → `MainWindow::onRunFinished()`: 游戏结束后返回主菜单。

3 小组成员分工

姓名	主要工作	具体内容
杨靖宇	战斗系统与效果框架	设计 Effect/Condition/Action 体系, 实现卡牌效果、伤害计算、能力触发
胡薛淇	敌人 AI 与跑团管理	编写所有敌人 AI、RunManager 楼层逻辑、祝福和选牌事件
泽仁罗布	UI 与动画	MainWindow 页面切换、Game 图形场景搭建、CardItem 交互、SpriteAnimator 动画系统
共同	测试与 Bug 修复	集成测试、数值平衡调整、代码审查

4 项目总结与反思

4.1 技术收获

- 深入实践了 C++ 多态与虚函数机制, 通过 `EffectAction` 和 `Condition` 纯虚基类实现了可高度扩展的效果系统。
- 理解了 Qt 信号槽机制在模块解耦中的重要性, 尤其是跨窗口和跨对象的通信。
- 体验了使用 `QGraphicsView` 框架搭建 2D 游戏界面的流程, 包括自定义 item、动画和悬浮提示。

4.2 不足与改进

- 只实现了楼层间的存档功能, 后续可完善战斗过程中的实时存档功能
- 敌人 AI 目前是基于固定回合模式的简单状态机, 可以考虑更灵活的决策树。
- 部分卡牌效果描述和图标欠打磨, 美术资源可进一步丰富。

4.3 协作体会

1 利用 Git 进行版本管理是高效并行开发、减少冲突的好方法。2 定期同步和整合对于维持代码风格一致非常重要。3 一个严谨的大框架是后续高效实现项目的基础, 在此之上, 明确的分工、定义接口先行也是项目落地的要诀。